VEHRON: a modular forward-time vehicle simulation framework for BEV energy, thermal, and subsystem studies

VEHRON Contributors

April 17, 2026

Abstract

VEHRON (VEHicle Research and Optimisation Network) is an open-source Python framework for forward-time road-vehicle simulation with an active focus on battery-electric passenger vehicles. The software is organized around a small orchestration core, a shared state bus, validated YAML configuration, and interchangeable subsystem modules for driver, longitudinal dynamics, drivetrain reduction, electric motor, inverter, regenerative braking, battery, HVAC, auxiliary loads, and low-order thermal trends. The current repository emphasizes engineering usability: reproducible case packaging, swappable battery and HVAC slots, fixed-step multi-rate execution, live run telemetry, and exported plots suitable for quick study review. This paper documents the present software architecture, the implemented model set, and representative outputs from highway and stop-start cycles. The manuscript is written as a software paper and preprint-style companion for eventual archival release and adaptation to Journal of Open Source Software submission requirements.

1 Summary

VEHRON is a component-based vehicle simulator designed for engineering studies such as energy consumption estimation, state-of-charge (SoC) trajectory prediction, thermal trend inspection, and comparison of operating cases across vehicle configurations. The project uses a forward-time simulation loop rather than a monolithic block diagram. A central software coordinator advances time and applies module outputs, while each module owns a bounded physical domain. In the current validated path, VEHRON assembles a battery-electric four-wheeler (BEV 4W) chain spanning speed tracking, longitudinal dynamics, drivetrain reduction, motor, inverter, regenerative braking, battery, HVAC, auxiliary loads, and low-order thermal management.

The repository has recently been extended from a mainly developer-oriented engineering MVP into a more usable run environment. The command-line entry point now prints a run "spec sheet" before simulation, streams live vehicle-level telemetry during execution at 10 s intervals, stores case packages under `output/cases/`, and generates post-run plots for motion, SoC with idle-state indication, and thermal channels. The full automated suite currently reports 25 passing tests in this workspace, including a runner integration test that checks the case package and plot outputs.

2 Statement of Need

Vehicle-simulation workflows often sit at an uncomfortable boundary. High-fidelity tools can be expensive, difficult to extend, or awkward to automate, while ad hoc spreadsheets and one-off scripts rarely scale into reusable research software. VEHRON is intended for users who need a transparent simulation stack that can be inspected, modified, and gradually upgraded without rewriting the whole loop each time a subsystem model changes. The current codebase is particularly aimed at:

- early-stage BEV sizing and trade studies,
- subsystem sensitivity checks for battery, HVAC, and auxiliary loads,
- repeatable comparison across route and environment cases,
- integration of proprietary battery or HVAC models without committing the proprietary source into the repository.

This last point is one of VEHRON's differentiators. The repository includes in-repo reference models, but the battery and HVAC slots are explicit interfaces. A third-party pack model can be loaded at runtime from a local Python file if it inherits from the VEHRON base class, allowing proprietary electro-thermal logic to run inside an otherwise open vehicle-level simulation workflow.

3 Software Architecture

The present software architecture follows four rules. First, `SimEngine` owns orchestration and time stepping only; it does not carry subsystem physics internally. Second, `SimState` is the shared state bus between modules. Third, each module returns only the outputs it computes. Fourth, vehicle hardware and mission definitions are provided through validated YAML configuration rather than ad hoc runtime code.

At a high level, one run proceeds as follows:

1. vehicle YAML and testcase YAML are loaded and validated,
2. boundary-unit conversions are applied at the loader edge,
3. `SimEngine` instantiates the configured modules,
4. the engine advances the master clock and schedules modules according to their execution divisors,
5. module outputs are written back onto `SimState`,
6. energy bookkeeping, logging, and termination checks are applied,
7. a flat time series and case package are emitted for post-processing.

This separation keeps the core runtime small while allowing subsystems to evolve independently. It also makes the flat time-series export natural rather than an afterthought, which is useful for study logging, pack-team exports, and case packaging.

3.1 Fixed-step multi-rate execution

VEHRON uses a fixed-step master clock with per-module execution divisors. If the master timestep is Δt and a module declares a divisor d, then its effective update interval is

$$\Delta t_{\text{eff}} = d\,\Delta t. \tag{1}$$

Fast domains such as the driver, longitudinal dynamics, motor, inverter, and regenerative braking run every master step in the active BEV chain, while slower domains such as battery electrical updates, HVAC, and thermal trends run less often. To avoid physically meaningless snapshots for slow modules, VEHRON calls `accumulate()` on every master step and `flush_accumulator()` immediately before the scheduled update of the slower module. This lets slow modules use interval-averaged or interval-summed inputs instead of a single instantaneous value.

3.2 Shared state contract

The current shared state contains the motion variables, battery variables, motor variables, key temperatures, power channels, and cumulative energy channels needed by the active BEV path. Important state conventions include SI internal units, Kelvin for temperature, positive battery current for discharge, positive battery power for battery-side discharge, and positive regenerative power for energy recovered into the battery boundary.

4 Implemented Models

4.1 Driver and longitudinal dynamics

The active driver model is a PID-style speed tracker. With speed error

$$e(t) = v_{\text{target}}(t) - v(t), \tag{2}$$

the controller command is

$$u(t) = K_p e(t) + K_i \int e(t)\, dt + K_d \frac{de(t)}{dt}, \tag{3}$$

which is split into non-negative throttle and brake requests by clamping positive and negative command branches separately.

The longitudinal dynamics module then evaluates the net force

$$F_{\text{net}} = F_{\text{trac}} - F_{\text{brake}} - F_{\text{aero}} - F_{\text{roll}} - F_{\text{grade}}, \tag{4}$$

with acceleration

$$a = \frac{F_{\text{net}}}{m}. \tag{5}$$

The current implementation represents aerodynamic drag as

$$F_{\text{aero}} = \frac{1}{2}\rho C_d A v_{\text{rel}}^2, \tag{6}$$

and rolling plus grade loads as low-order functions of mass, road grade, and speed. This is a simple but effective road-load closure for whole-vehicle energy studies.

4.2 Motor, inverter, and regenerative braking

The default analytical motor model clamps torque and mechanical power to the configured peak limits and computes a low-order efficiency estimate from load fraction and speed fraction. For motoring operation, electrical drive demand is estimated from mechanical power and the motor efficiency, then passed through an inverter stage with configurable efficiency. Regenerative braking is modeled as a blended brake controller that limits recovered power by both the requested braking level and the instantaneous kinetic power available at the wheel boundary.

4.3 Battery models

VEHRON currently carries two in-repo battery models. The baseline `Rint` model provides lightweight SoC and terminal-voltage behavior suitable for quick energy studies. The higher-fidelity two-RC ECM model introduces an open-circuit voltage estimate and two polarization branches, following the widely used equivalent-circuit modeling family for lithium-ion batteries [2, 5]. Conceptually,

$$V_{\text{term}} = V_{\text{ocv}} - IR_0 - V_{\text{rc1}} - V_{\text{rc2}}. \tag{7}$$

Each RC branch relaxes toward its current-dependent equilibrium with its own time constant,

$$\tau_i = R_i C_i. \tag{8}$$

The current implementation uses a stable exponential update for these branch states. Both in-repo battery models expose SoC, battery current, terminal voltage, and net battery power. Both also accept optional external charging power as an input channel, though a full automatic stop-charge-resume mission controller has not yet been implemented in the runtime.

The battery slot is a formal interface rather than a hard-coded in-repo dependency. This means a proprietary pack model can be loaded from a local Python file at runtime as long as it inherits from the VEHRON battery base class. The same approach exists for HVAC.

4.4 Cabin HVAC and thermal trends

The cabin HVAC model is a low-order lumped thermal model aligned with the standard vehicle-cabin thermal-model family used for energy studies [1, 3, 4, 6]. The present formulation decomposes the thermal load into envelope exchange, solar gain, ventilation or fresh-air load, occupant sensible load, and HVAC-delivered heat:

$$Q_{\text{net}} = Q_{\text{envelope}} + Q_{\text{solar}} + Q_{\text{ventilation}} + Q_{\text{occupants}} + Q_{\text{hvac}}. \tag{9}$$

Cabin temperature then evolves according to

$$\frac{dT_{\text{cabin}}}{dt} = \frac{Q_{\text{net}}}{C_{\text{total}}}, \tag{10}$$

and HVAC electrical power is estimated from the thermal load and the relevant coefficient of performance. Battery, motor, and coolant temperatures are presently represented by low-order trend models rather than high-resolution thermal networks. This is a deliberate trade-off to keep the whole-vehicle loop fast while still exposing thermal behavior at the vehicle level.

5 Representative Workflows and Outputs

The command-line workflow now exposes more of the run to the user than the earlier CLI summary. Before execution, VEHRON prints a test specification sheet containing the vehicle, testcase, route mode, environment, battery configuration, and simulation settings. During execution, it prints vehicle-level progress at 10 s intervals, including distance traveled, speed, acceleration, SoC, and available temperatures such as ambient, motor, battery, coolant, and cabin temperatures. After execution, the runtime writes a case package containing:

- `summary.json`,
- `timeseries.csv`,
- copied input YAML files,
- resolved YAML snapshots,
- generated plots,
- a brief case README.

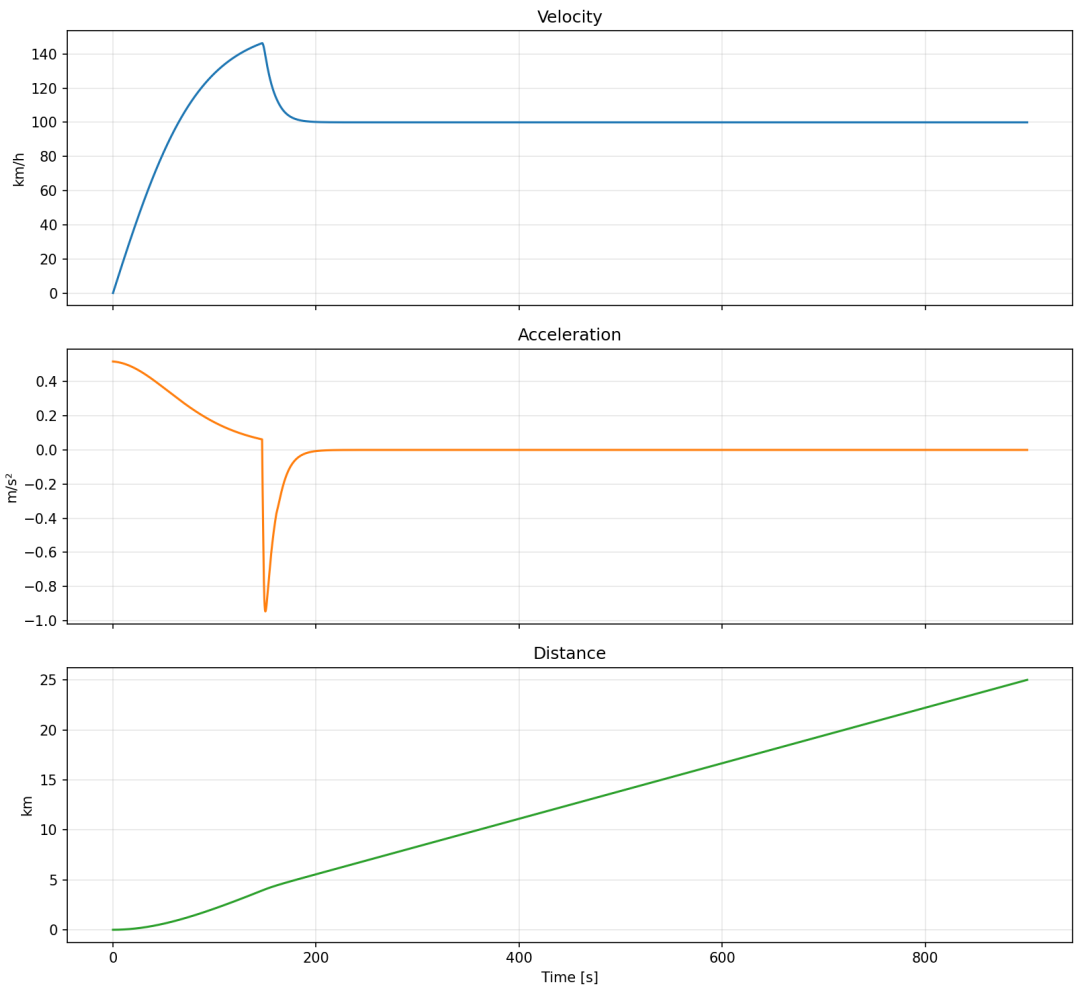

Figure 1: Motion channels from the default flat-highway BEV case. The case package includes velocity, acceleration, and distance traces together in one plot.

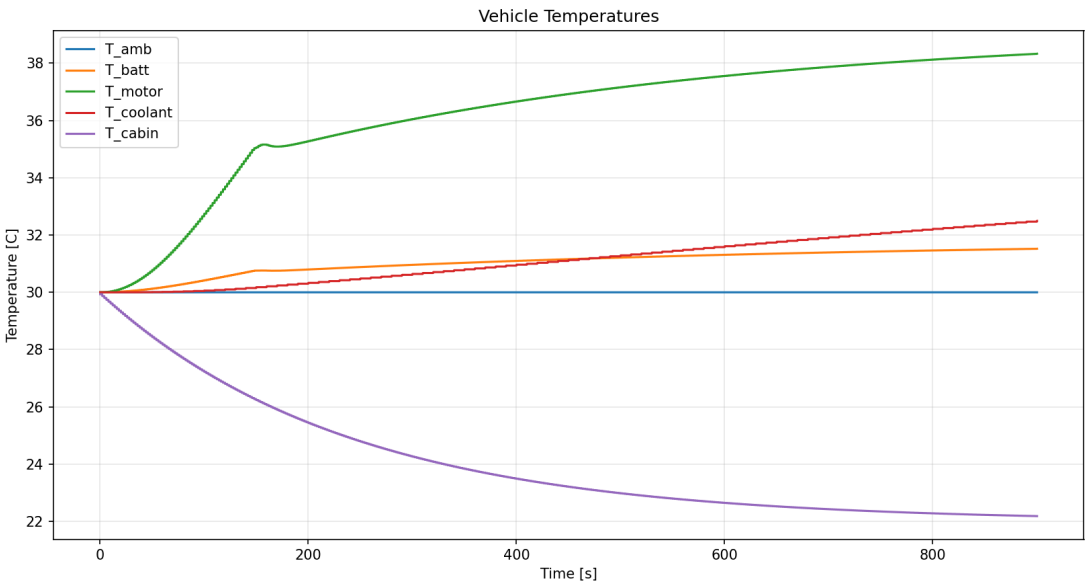

Figure 2: Vehicle temperature channels from the same highway case. The current built-in plots expose ambient, battery, motor, coolant, and cabin temperature trajectories.

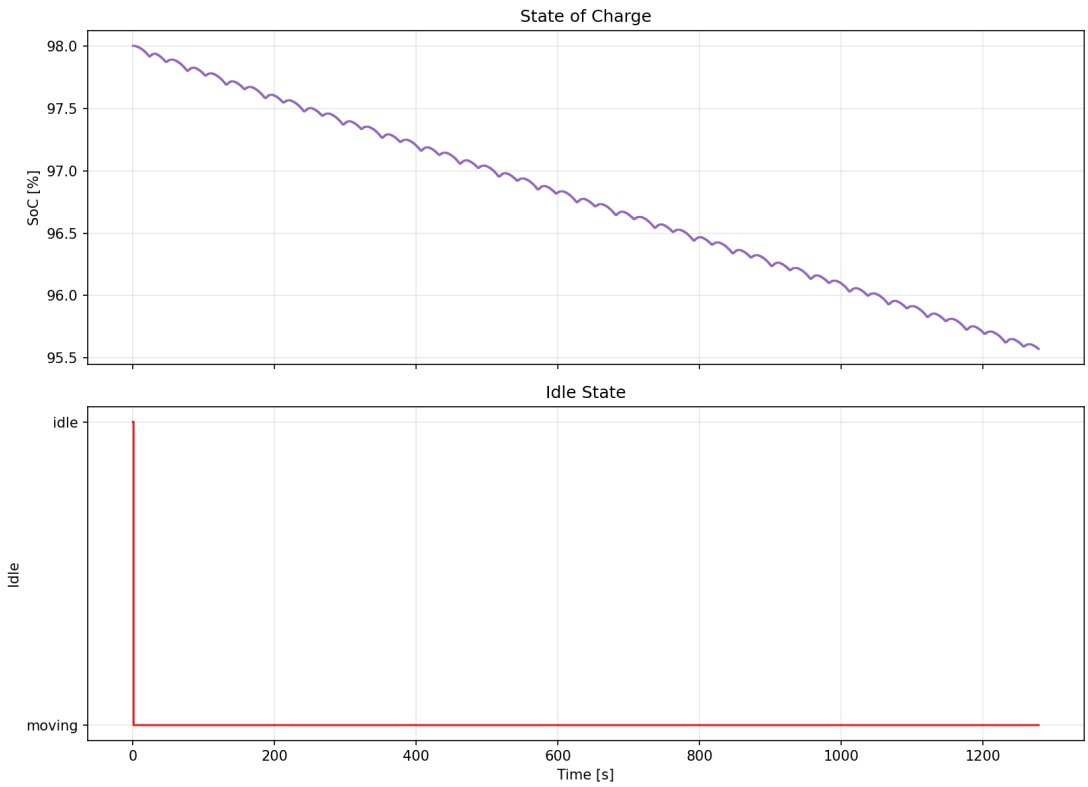

Figure 3: SoC and idle-state plot for the WLTP-like stop-start city cycle. This case shows repeated regenerative events and a smaller net SoC drop than the flat-highway case.

Two current reference runs illustrate the present behavior. The flat-highway 100 km h^{-1} BEV sedan case covers 25.0 km in about 900 s, with a SoC drop from 0.98 to about 0.887 and a net energy use of roughly 5.10 kW h. The WLTP-like stop-start city case covers just over 8.0 km with a SoC drop from 0.98 to about 0.956 and accumulates substantially more regenerative

recovery, which is visible in the city-cycle SoC behavior and the regenerated-energy total.

Table 1: Representative current case summaries generated by VEHRON case packages.

Case	Distance (km)	Final SoC	Energy (Wh)	Regen (Wh)	Final T_{motor} (C)
Flat highway 100 km/h	25.001	0.8872	5103.6	33.4	38.33
City WLTP-like stop-start	8.001	0.9557	1337.1	691.8	31.27

6 Current Limitations and Next Steps

VEHRON is still an engineering MVP. The in-repo battery models are low-order pack models rather than full electrochemical solvers. Thermal behavior is tracked at pack, motor, coolant, and cabin trend level rather than through resolved cell-core or condenser and radiator models. The active route inputs are parametric cases and CSV drive cycles; a GPX-derived mission path discussed during this development session would be a natural next addition, ideally followed by an inverse-replay mode that derives wheel demand directly from measured motion rather than from a controller tracking a reference.

The runtime now supports case packaging, live telemetry, plots, and export hooks, but several roadmap items remain important:

- a mission controller for automatic stop-charge-resume behavior,
- richer standardized thermal telemetry for pluggable devices,
- GPX route ingestion and trace replay,
- stronger energy-balance and end-to-end integration checks beyond the current basic integration tests,
- final alignment with JOSS submission packaging, which uses `paper.md` rather than LaTeX as the formal submission format.

7 Availability

VEHRON is distributed as Python source under the AGPL-3.0-only license in this repository. The present workspace includes automated tests, example archetypes and testcases, case-package generation, and a manuscript folder with generated figures. The code is intended to be archived with a DOI-bearing software release for eventual Zenodo deposition and adapted into the exact Journal of Open Source Software submission format when the project is ready for formal review.

8 Acknowledgements

The project reflects sustained personal engineering effort toward an open, modular vehicle-simulation stack that can host both in-repo and externally supplied subsystem models. The recent CLI and reporting improvements documented here were developed directly against the current repository and validated with the local test suite.

References

[1] Anonymous. A systematic review on modelling the thermal environment of vehicle cabins. *Applied Thermal Engineering*, 257:124142, 2024. doi: 10.1016/j.applthermaleng.2024.124142.

[2] Xiaosong Hu, Shengbo Eben Li, and Huei Peng. A comparative study of equivalent circuit models for Li-ion batteries. *Journal of Power Sources*, 198:359–367, 2012. doi: 10.1016/j.jpowsour.2011.10.013.

[3] D. Marcos, F. J. Pino, C. Bordons, and J. J. Guerra. The development and validation of a thermal model for the cabin of a vehicle. *Applied Thermal Engineering*, 66(1-2):646–656, 2014. doi: 10.1016/j.applthermaleng.2014.02.054.

[4] U. Noreen, M. Farooq, and M. Batool. Thermal management of vehicle cabins, external surfaces, and onboard electronics: An overview. *Engineering*, 5(5):954–969, 2019. doi: 10.1016/j.eng.2019.02.009.

[5] Jie Su, Maosong Lin, Shunli Wang, Jin Li, James Coffie-Ken, and Fei Xie. An equivalent circuit model analysis for the lithium-ion battery pack in pure electric vehicles. *Measurement and Control*, 52(3-4):193–201, 2019. doi: 10.1177/0020294019827338.

[6] B. Torregrosa-Jaime, F. Bjurling, J. M. Corberan, and F. Di Sciullo. Transient thermal model of a vehicle's cabin validated under variable ambient conditions. *Applied Thermal Engineering*, 75:45–53, 2015. doi: 10.1016/j.applthermaleng.2014.05.074.